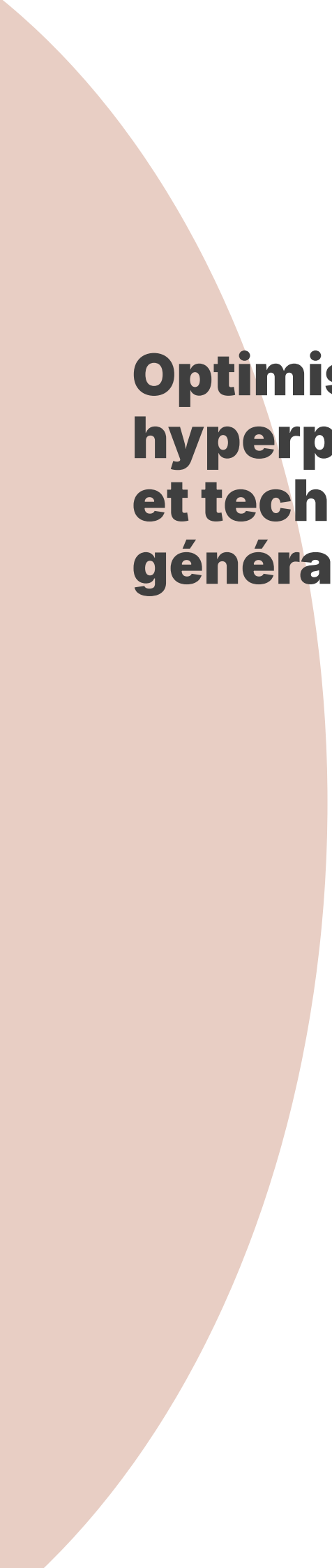




Optimisation des hyperparamètres et techniques de généralisation

Table des matières

I - Contexte	3
II - Fondamentaux de l'optimisation des hyperparamètres	4
A. Cadre méthodologique.....	4
B. Méthodes d'exploration de l'espace des hyperparamètres	5
III - Implémentation et automatisation de l'optimisation	8
A. Techniques avancées d'optimisation.....	8
B. Infrastructure et bonnes pratiques.....	9
IV - Techniques de régularisation et contrôle du surapprentissage	11
A. Principes et méthodes de régularisation	11
B. Adaptation et équilibrage des techniques de régularisation	12
V - Synthèse	14
VI - Auto-évaluation	18
VII - Exercice : Optimiser un modèle d'analyse de sentiment client	22
Solutions des exercices	24

I Contexte

Contexte

Dans la pratique de l'intelligence artificielle, un algorithme performant ne se limite pas à l'innovation de son architecture ou à la quantité de données disponibles. Les véritables gains de performance, de robustesse et de pertinence se jouent à travers deux grands leviers : **l'optimisation des hyperparamètres** et la maîtrise des **techniques de régularisation**.

Les **hyperparamètres** sont des réglages externes à l'apprentissage du modèle, fixés manuellement ou *via* des stratégies automatisées, qui déterminent en profondeur le comportement du modèle : nombre de couches d'un réseau de neurones, taux d'apprentissage, taille des lots de données (batch), critères de régularisation... Leur paramétrage influe directement sur la capacité du modèle à résoudre la tâche cible, sa vitesse de convergence ou sa stabilité. Les méthodes d'optimisation varient : du grid search le plus classique à l'optimisation bayésienne avancée, en passant par les algorithmes évolutionnaires ou adaptatifs, chaque famille de modèles et chaque contexte opérationnel exige une approche spécifique pour explorer efficacement cet « *espace de réglage* » souvent immense et multidimensionnel.

La **régularisation** est quant à elle indispensable pour éviter que le modèle n'apprenne « *trop bien* » les données d'entraînement, au point de devenir incapable de généraliser à des cas nouveaux : c'est le surapprentissage (overfitting). Elle agit en imposant une contrainte supplémentaire (pénalité sur la complexité, bruit ou perturbation volontaire, contrôle précoce de l'arrêt d'apprentissage, etc.), forçant ainsi le modèle à privilégier des solutions sobres, résilientes et applicables en production. Les techniques varient de la pénalisation L1/L2 (qui simplifient la structure du modèle), à des méthodes comme le dropout, la batch normalization, l'augmentation de données ou les ensembles (ensemble learning). Leur choix dépend du type de modèle, du volume de données, des exigences métier ou des contraintes industrielles rencontrées.

Sur le terrain, ces deux volets sont indissociables : réussir à optimiser les hyperparamètres sans une régularisation adéquate mène souvent à un modèle brillant sur le papier mais catastrophique en production ; trop régulariser peut « casser » le potentiel du modèle. Les meilleures équipes data naviguent ce compromis par une expérimentation méthodique, des protocoles de validation rigoureux, et l'automatisation intelligente du tuning au sein de leur workflow.

Qu'il s'agisse de prédire la demande dans le retail, d'optimiser des modèles de diagnostic médical, de renforcer la robustesse des systèmes de recommandation ou d'améliorer le scoring de risque dans la finance, ce couple optimisation-régularisation est l'un des piliers techniques de l'IA moderne, conditionnant sa fiabilité et son adoption métier à grande échelle.

II Fondamentaux de l'optimisation des hyperparamètres

A. Cadre méthodologique

Rôle et impact des hyperparamètres

L'optimisation des **hyperparamètres** est un pilier fondamental pour obtenir des modèles d'IA robustes et performants. Un hyperparamètre est un réglage externe au modèle : contrairement aux paramètres ajustés lors de l'apprentissage (poids, biais, etc.), il est fixé par l'ingénieur ou par le système d'optimisation avant l'entraînement. Il peut s'agir du taux d'apprentissage, du nombre de couches ou de neurones dans un réseau, de la profondeur d'un arbre de décision, du coefficient de régularisation ou du type de kernel utilisé pour un SVM.

Leur impact est direct : ils gouvernent la capacité d'apprentissage, la rapidité de convergence, la stabilité et surtout l'aptitude du modèle à généraliser sur des données nouvelles. Un mauvais choix provoque soit du sous-apprentissage (modèle trop faible ou trop bridé), soit du surapprentissage (modèle trop « collé » à l'historique), ou des difficultés d'optimisation (divergence, plateau).

On distingue plusieurs familles :

- Les **hyperparamètres architecturaux** (choix de la structure, nombre de couches, activation, etc.)
- Les **hyperparamètres d'optimisation** (learning rate, batch size, etc.)
- Les **hyperparamètres de régularisation** (coefficients L1/L2, dropout, etc.)

La réelle difficulté réside dans la **dimension multidimensionnelle et non linéaire** de l'espace à explorer, où des interactions complexes rendent toute recherche exhaustive rapidement impossible, surtout sur des modèles avancés ou des jeux volumineux.

Paramètres vs Hyperparamètres

Az Définition

Les **paramètres** sont les valeurs internes du modèle apprises automatiquement (poids et biais d'un réseau de neurones).

Les **hyperparamètres** sont les valeurs fixées avant/après entraînement par l'ingénieur ou une stratégie de recherche (learning rate, batch size, profondeur d'arbre...).

Les premiers s'ajustent à partir des données, les seconds orientent **la manière d'apprendre**.

Définition d'une stratégie d'optimisation cohérente

Optimiser les hyperparamètres exige de structurer sa démarche : il faut d'abord choisir les objectifs d'optimisation (précision, généralisation, temps d'inférence, etc.), sélectionner les hyperparamètres les plus influents à ajuster (pour éviter l'explosion combinatoire), fixer les métriques d'évaluation pertinentes (score, AUC, F1, coût calculatoire...) et planifier l'utilisation des ressources de calcul (nombre d'expériences, parallélisation, temps maximal, etc.).

Des exemples industriels montrent que les entreprises performantes automatisent ces choix : elles commencent par une analyse d'importance, établissent un protocole expérimental avec sauvegarde systématique des essais et hiérarchisent les recherches (d'abord les grands leviers, puis les réglages fins).

Selon le type de modèle, certains hyperparamètres sont systématiquement prioritaires : nombre d'arbres et profondeur pour les forêts aléatoires, learning rate et batch size pour les réseaux, coefficient C et gamma pour les SVM...

Cas de la finance - surfitting réglementaire

👁 Exemple

Dans une banque française, un tuning trop agressif des hyperparamètres d'un score crédit a mené à un modèle ultra-performant sur dataset interne, mais désastreux en audit réglementaire car il généralisait mal (sur-ajustement). Après analyse, seuls 3 hyperparamètres principaux ont été retenus et ré-optimisés : learning rate, régularisation L2, profondeur max des arbres.

Risque de coût computationnel et de surajustement

⚠ Attention

Une exploration trop large ou mal guidée peut consommer un temps machine considérable sans gain réel : d'où l'intérêt de restreindre le périmètre aux hyperparamètres véritablement déterminants, ou d'utiliser des méthodes avancées pour guider la recherche.

Spécificités selon les types de modèles

Chaque famille d'algorithme possède des hyperparamètres critiques spécifiques :

- Pour un arbre de décision, la profondeur maximale, le nombre minimal d'échantillons par feuille ou le critère de division
- Pour un SVM, le kernel, ses coefficients associés et le paramètre de pénalisation
- Pour un réseau de neurones, le nombre de couches, le type d'activation, le learning rate, le batch size, et le schéma de régularisation

Dans la finance, optimiser les paramètres de régularisation pour éviter du surajustement sur des données bruitées est souvent prioritaire. En santé, la stabilité et la capacité à généraliser sont centrales, d'où un focus sur la robustesse du tuning et la validation croisée stricte. Dans le retail, le compromis vitesse-précision est souvent dimensionnant (modèle déployé en temps réel sur applis mobiles par exemple).

Documentez chaque essai d'optimisation

👤 Conseil

Pour chaque combinaison testée, notez systématiquement : les valeurs d'hyperparamètres, la métrique obtenue, le temps de calcul, l'environnement de test. Cette traçabilité permet de revenir sur les essais fructueux ou défaillants, de collaborer efficacement, et d'expliquer votre démarche lors d'un audit.

B. Méthodes d'exploration de l'espace des hyperparamètres

Approches par recherche exhaustive et aléatoire

Les deux méthodes les plus basiques pour explorer l'espace des hyperparamètres sont la **recherche par grille** (grid search) et la **recherche aléatoire** (random search).

La recherche par grille teste **toutes les combinaisons possibles** dans un ensemble prédéfini de valeurs pour chaque hyperparamètre. Elle garantit de trouver la meilleure combinaison **dans l'espace testé**, mais devient vite coûteuse dès que le nombre de paramètres ou de valeurs augmente (effet dit « *combinatoire exponentiel* »).

La recherche aléatoire choisit **des combinaisons au hasard** dans les plages définies : moins exhaustive mais souvent plus efficace en pratique, car les hyperparamètres influents ne sont pas tous également sensibles à leur plage de variation. Les études de Bergstra & Bengio (2012) ont montré que, pour un même budget d'essais, la recherche aléatoire retrouve souvent des zones optimales plus vite que la grille.

Par exemple, pour un modèle de prévision de demande, où seuls le nombre d'arbres et le learning rate influent fortement, un random search sur des plages larges repère rapidement la bonne zone, tandis qu'un grid search testerait inutilement toutes les combinaisons de paramètres peu influents.

Effet combinatoire

Az Définition

L'**effet combinatoire** apparaît quand le nombre de combinaisons d'hyperparamètres explose : 5 paramètres x 10 valeurs chacun = 100 000 essais possibles. Ce phénomène rend la grid search impraticable dans la plupart des problèmes réels.

Limites de la recherche exhaustive

⚠ Attention

La grid search est tentante pour sa promesse de couverture complète, mais elle gaspille souvent des ressources en testant de nombreuses combinaisons peu utiles. Elle est surtout adaptée à de petits espaces bien délimités.

Méthodes évolutives et par population

Inspirées des processus biologiques de sélection naturelle, ces méthodes font évoluer **une population de configurations d'hyperparamètres** à travers des opérations d'héritage, de mutation et de sélection.

Les **algorithmes génétiques**, **CMA-ES** et autres stratégies évolutionnaires explorent l'espace de manière large tout en affinant les individus prometteurs.

L'avantage est une bonne capacité à échapper aux *minima* locaux et à travailler sur des espaces vastes et non convexes.

Par exemple, dans un système de recommandation e-commerce, un algorithme évolutionnaire a optimisé simultanément 15 hyperparamètres continus et discrets sans recours à un modèle différentiable, obtenant un gain de 6 % sur les taux de clics.

Optimisation multi-objectifs

Az Définition

L'optimisation **multi-objectifs** consiste à rechercher un compromis entre plusieurs métriques parfois contradictoires (ex : maximiser la précision tout en minimisant le temps d'inférence et la consommation mémoire). Les solutions optimales forment alors ce qu'on appelle un **front de Pareto** : aucun objectif ne peut être amélioré sans en dégrader au moins un autre.

Stratégies d'optimisation multi-objectifs

Dans un contexte industriel, il est rare qu'un seul critère compte. On peut vouloir un modèle très performant mais aussi léger et rapide. Les stratégies multi-objectifs évaluent simultanément plusieurs métriques et conservent un ensemble de configurations « *non dominées* ».

Par exemple, pour un modèle embarqué sur mobile de détection vocale, le front de Pareto permet à l'équipe d'offrir plusieurs configurations : l'une plus précise mais un peu plus lente, l'autre très rapide avec un léger compromis sur la précision.

Pareto optimalité en industrie automobile

[👁 Exemple](#)

En conception de modèles prédictifs embarqués dans des véhicules, les ingénieurs recherchent le compromis entre :

- **Précision maximale** (sécurité des détections)
- **Faible temps d'inférence** (temps réel)
- **Faible consommation énergétique**

Le front de Pareto aide à identifier les architectures qui optimisent ce compromis selon chaque usage.

Exploration séquentielle et adaptative

Ces méthodes adaptent la recherche **en fonction des résultats précédents**. Plutôt que de définir tous les essais à l'avance, elles ajustent dynamiquement plages et priorités selon les observations.

Par exemple :

- **TPE (Tree-structured Parzen Estimator)**, qui modélise séparément les zones de bons et mauvais résultats pour diriger les essais.
- Recherche bayésienne adaptative, qui affine ses prédictions itératives sur les régions prometteuses.

Pour un assistant vocal francophone, une start-up a utilisé une approche séquentielle TPE, réduisant le temps total d'optimisation de 40 % tout en améliorant le taux de reconnaissance de 3 points par rapport à un random search limité.

Combiner les approches selon les phases

[👤 Conseil](#)

Commencez souvent par un échantillonnage large (random ou évolutif) pour explorer l'espace, puis affinez sur la zone prometteuse avec une optimisation bayésienne ou adaptative : cette combinaison équilibre couverture et efficacité.

III Implémentation et automatisation de l'optimisation

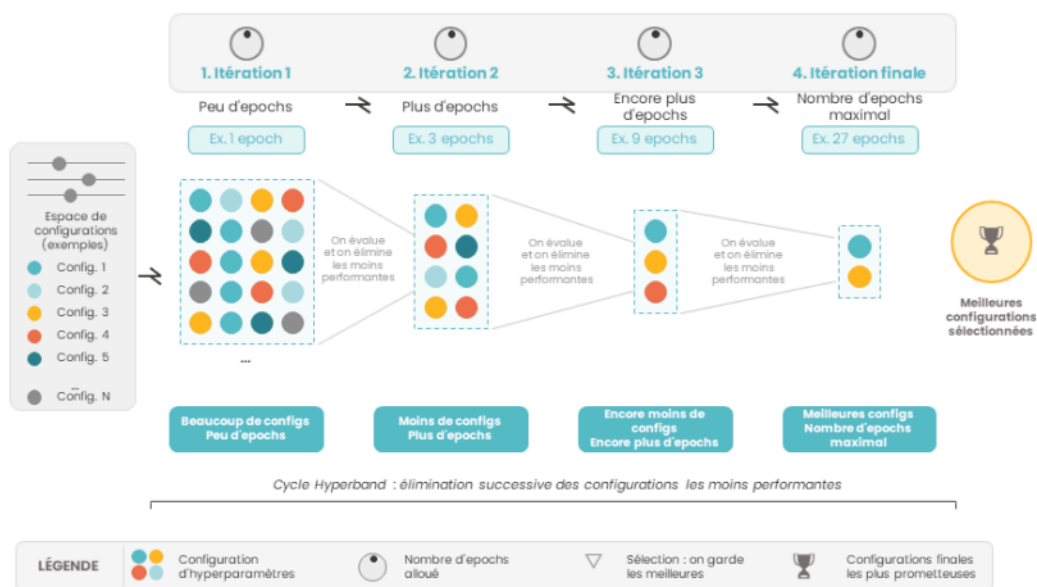
A. Techniques avancées d'optimisation

Optimisation multi-fidélité et arrêt précoce intelligent

Lorsque l'espace des hyperparamètres à explorer est large, une recherche exhaustive devient vite hors de portée en termes de budget temps ou ressources. Les approches dites **multi-fidélité** consistent à estimer rapidement la performance potentielle d'une configuration sans l'entraîner jusqu'au bout. Par exemple, on peut interrompre l'entraînement après quelques itérations et n'allouer du temps complet qu'aux configurations prometteuses.

Des protocoles comme **Hyperband** ou **Successive Halving** organisent ainsi la compétition entre jeux d'hyperparamètres : seules les configurations les plus performantes à un stade intermédiaire passent au tour suivant. Cette sélection progressive permet un gain significatif d'efficacité, en particulier lorsque chaque entraînement est coûteux.

Dans des contextes industriels où les cycles d'entraînement peuvent durer plusieurs heures, combiner un arrêt précoce intelligent avec la réallocation dynamique des ressources aux candidats les plus pertinents permet de diviser le temps de recherche par deux, tout en conservant un haut niveau de performance finale.



Optimisation des hyperparamètres spécifiques aux architectures complexes

Les architectures profondes comme les **réseaux convolutifs**, les **RNN** ou les **transformers** demandent des réglages plus fins et souvent interdépendants : taille et nombre de couches, paramètres des blocs d'attention, stratégie de normalisation, etc. L'efficacité de l'optimisation dans ces cas passe par une sélection judicieuse de quelques variables clés, puis par des ajustements progressifs.

Pour un réseau transformer par exemple, explorer simultanément le nombre de têtes d'attention, la dimension cachée et le dropout peut être priorisé avant d'affiner d'autres paramètres secondaires. Ce ciblage préalable permet de bénéficier d'un premier gain substantiel avant d'entrer dans des ajustements plus coûteux.

Effet domino des réglages dans les modèles complexes



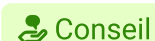
Dans les architectures profondes, la modification d'un hyperparamètre structurant peut impacter plusieurs autres réglages de façon indirecte. Un ajustement du batch size peut nécessiter de revoir le learning rate, et un changement de dimension cachée peut exiger d'adapter la stratégie de régularisation.

Recherche d'architecture neuronale (NAS)

Les méthodes de **Neural Architecture Search** automatisent le processus de découverte de combinaisons optimales entre structure de réseau et réglages, en testant un très grand nombre de variantes via des algorithmes de recherche guidée. Certaines approches utilisent le **renforcement** pour apprendre quelles architectures produisent les meilleurs gains, d'autres des méthodes **différentiables** pour ajuster la topologie en continu.

Ce type de recherche reste coûteux, mais dans certains domaines à forts enjeux (vision médicale, NLP industriel), il permet de découvrir des architectures inédites surpassant largement les configurations conçues manuellement.

Utilisez NAS avec parcimonie



Le NAS consomme énormément de ressources (GPU/TPU). Il est surtout recommandé pour des contextes stratégiques (santé, industrie critique). Pour des projets métiers classiques (retail, RH), une optimisation guidée (Bayésienne/TPE) suffit largement.

B. Infrastructure et bonnes pratiques

Mise en place d'une infrastructure d'optimisation efficace

La performance d'un processus d'optimisation repose autant sur les méthodes employées que sur **l'infrastructure** permettant de les exécuter efficacement. Déployer une architecture de calcul distribuée ou mutualisée permet de paralléliser les essais, d'exploiter au mieux GPU ou TPU, et d'accélérer drastiquement le cycle d'expérimentation.

L'organisation doit aussi prévoir :

- Un suivi centralisé des métriques,
- Un stockage versions des configurations,
- Une planification des tâches qui minimise l'attente et les coûts.

L'expérience d'entreprises ayant industrialisé leur tuning montre que le gain de vitesse et la qualité des résultats sont démultipliés lorsque ces aspects sont pensés dès le départ plutôt que greffés a posteriori.

Bonnes pratiques de validation et d'évaluation

Une optimisation mal contrôlée peut conduire à un modèle surajusté... aux données de validation. Les bonnes pratiques comprennent :

- La **validation croisée imbriquée** pour séparer l'étape de sélection des hyperparamètres de l'évaluation finale,
- Un suivi des performances sur des **jeux indépendants** afin de confirmer la généralisation,
- La mise en place de **bornes raisonnables** pour éviter les configurations extrêmes, parfois séduisantes sur un jeu mais désastreuses sur un autre.

Cette rigueur évite les illusions de performance qui s'effondrent en production.

Préserver un échantillon



Ce jeu, mis de côté dès le début, ne doit être utilisé qu'une fois, à la toute fin, pour mesurer l'efficacité réelle de l'optimisation et garantir une estimation non biaisée.

Documentation et reproductibilité des optimisations

Chaque campagne d'optimisation doit être **documentée** : valeurs testées, résultats obtenus, environnement de calcul, version des datasets. Des outils comme Mlflow, Weights & Biases ou DVC offrent un suivi automatisé et facilitent la collaboration autant que les audits.

La reproductibilité passe par :

- La conservation des seeds
- Le gel des bibliothèques utilisées
- L'archivage des scripts et configurations

Dans des environnements réglementés (banque, santé), cette exigence devient un prérequis pour toute mise en production.

IV Techniques de régularisation et contrôle du surapprentissage

A. Principes et méthodes de régularisation

Concepts fondamentaux de la régularisation

La régularisation est un **levier central** pour améliorer la capacité de généralisation d'un modèle et limiter le surapprentissage (overfitting). Elle consiste à introduire dans l'entraînement des contraintes ou des pénalités qui **limitent la complexité effective du modèle**, le forcent à apprendre des représentations plus sobres et stables.

Un modèle trop complexe, surtout sur un jeu de données limité ou bruité, a tendance à « *mémoriser* » ses exemples d'entraînement, produisant de belles performances internes mais des échecs sur les données nouvelles. La régularisation agit sur ce **compromis biais-variance**, en acceptant une légère augmentative du biais pour réduire fortement la variance et donc améliorer la robustesse prédictive.

Dans la pratique, on distingue :

- Les techniques agissant directement sur la **structure ou la taille du modèle** (réduction du nombre de paramètres, pénalités L1/L2).
- Les méthodes introduisant de la **stochasticité contrôlée** (dropout, augmentation de données) pour éviter de sur-apprendre des patterns spécifiques au jeu d'entraînement.
- Les approches influençant la **dynamique d'entraînement** pour éviter la surexploitation de certaines configurations (early stopping, normalisations diverses).

Biais-Variance

Az Définition

Un **biais élevé** donne un modèle trop simple. On parle de sous-apprentissage.

Une **variance élevée** provoque un modèle trop complexe/sensible. On parle alors de sur-apprentissage.

La régularisation cherche un **équilibre optimal**.

Régularisation pour les réseaux de neurones

Les architectures neuronales comportant de nombreux paramètres, elles sont particulièrement sensibles au surapprentissage. Plusieurs méthodes éprouvées permettent de juguler ce risque :

- **Dropout** : désactivation aléatoire d'une proportion de neurones à chaque itération, ce qui force le réseau à apprendre des représentations plus distribuées.
- **Batch normalization** et **Layer normalization** : normalisation des activations à différentes étapes pour stabiliser l'apprentissage.
- **Régularisation spectrale** : contrôle explicite de la norme des poids pour limiter la capacité maximale du réseau.

Dans la vision par ordinateur, le dropout appliqué aux couches entièrement connectées après l'extraction de features a montré un fort impact sur la robustesse, tandis qu'en NLP les combinaisons de layer norm et dropout sont devenues un standard dans les transformers.

Stratégies avancées de lutte contre le surapprentissage

Au-delà des approches classiques, certaines stratégies exploitent **la perturbation volontaire des données ou du modèle** pour améliorer la généralisation :

- **Noise injection** : ajout de bruit léger dans les entrées, les activations ou les gradients.
- **Data augmentation** : variations artificielles des données (rotations, translations, synonymes, etc.) pour élargir l'espace d'entraînement.
- **Mixup** : interpolation linéaire entre deux exemples et leurs labels pour rendre la frontière de décision plus lisse.
- **Ensembles de modèles** : combinaison des prédictions de plusieurs modèles diversifiés pour réduire la variance.

Ces techniques sont utilisées dans les contextes critiques (imagerie médicale, détection de fraude) où la stabilité face à des données dégradées ou inédites est indispensable.

Data augmentation en e-commerce

[👁 Exemple](#)

Un site français de mode a enrichi son dataset en générant des images : rotations, éclairages modifiés, couleurs artificielles. Cette **augmentation de données** a réduit de 20 % l'overfitting du modèle de classification de vêtements.

B. Adaptation et équilibrage des techniques de régularisation

Sélection des techniques de régularisation selon le contexte

Le choix de la régularisation pertinente dépend de plusieurs facteurs :

- **Taille et qualité du dataset** : un petit jeu de données requiert souvent plus de régularisation et un usage intensif de l'augmentation de données.
- **Complexité du modèle** : plus un modèle est grand, plus il risque d'overfitting et nécessite des techniques ciblées.
- **Contraintes métier** : certains secteurs, comme la santé, privilégient la robustesse et l'explicabilité à la seule performance brute.

Il est donc essentiel d'éviter l'approche universelle et d'évaluer chaque technique à la lumière des contraintes opérationnelles.

Adaptation dynamique de la régularisation

Certaines approches adaptent l'intensité de la régularisation au fil de l'entraînement :

- **Dropout adaptatif** : taux de dropout évoluant en fonction des phases d'apprentissage,
- **Schedules de régularisation** : coefficients L1/L2 ajustés automatiquement selon les métriques,
- **Curriculum d'augmentation de données** : début avec des perturbations légères, puis intensification progressive pour renforcer la robustesse.

Dans des modèles soumis à des flux de données évolutifs, cette adaptation dynamique conserve la capacité d'apprentissage tout en maintenant la régularisation au bon niveau.

Évaluation de l'efficacité des stratégies de régularisation

Il ne suffit pas d'appliquer une technique : il faut la **mesurer**. L'évaluation passe par :

- Les **courbes d'apprentissage** (entraînement et validation) pour détecter les signes précoces d'overfitting,
- Les **tests de robustesse** sur des jeux modifiés ou contenant du bruit,
- La mesure de la **complexité effective** du modèle (ex : norme des poids, stabilité des prédictions sur répétitions).

Un suivi régulier permet d'ajuster la régularisation en fonction de l'évolution des performances et de la nature des données entrantes.

Sur-régularisation

⚠ Attention

Une régularisation excessive peut conduire à un modèle sous-expressif, incapable de capter les tendances importantes. Si le gap entraînement/validation est faible mais que les deux performances sont mauvaises, c'est souvent un signal de sous-apprentissage lié à trop de régularisation.

Régularisation et robustesse aux perturbations

Certaines méthodes augmentent non seulement la généralisation mais aussi la **résilience** aux données dégradées :

- Augmentation de données bruitées,
- Entraînement adversarial léger pour habituer le modèle à des entrées « *difficiles* »,
- Régularisation par entropie pour encourager des prédictions moins tranchées en cas d'incertitude.

Dans l'industrie, un projet de maintenance prédictive a utilisé ces approches pour rendre le modèle moins sensible aux valeurs aberrantes issues de capteurs défectueux, améliorant ainsi la continuité opérationnelle.

V Synthèse

A. Flashcards

Hyperparamètre

Réglage externe au modèle, fixé avant l'apprentissage, qui gouverne la manière d'apprendre.

Grid search

Méthode testant toutes les combinaisons possibles d'hyperparamètres dans un espace prédéfini.

Random search

Exploration aléatoire de l'espace des hyperparamètres, souvent plus efficace que la grille.

Optimisation bayésienne

Méthode guidant la recherche d'hyperparamètres par modélisation probabiliste des résultats antérieurs.

Surapprentissage (overfitting)

Phénomène où le modèle mémorise les données d'entraînement sans généraliser aux nouvelles.

Régularisation

Ensemble de contraintes limitant la complexité du modèle pour améliorer sa généralisation.

Dropout

Désactivation aléatoire de neurones durant l'entraînement pour forcer des représentations distribuées robustes.

Pénalité L1/L2

Contrainte ajoutée à la fonction de coût pour limiter la norme des poids.

Early stopping

Arrêt de l'entraînement dès que la performance sur validation cesse de s'améliorer.

Compromis biais-variance

Équilibre entre sous-apprentissage (biais élevé) et surapprentissage (variance élevée) pour optimiser la généralisation.

Batch normalization

Normalisation des activations par couche pour stabiliser et accélérer l'apprentissage.

Augmentation de données

Génération de variations artificielles des données pour élargir l'espace d'entraînement.

Hyperband

Méthode multi-fidélité arrêtant précocement les configurations d'hyperparamètres peu prometteuses.

Neural Architecture Search (NAS)

Recherche automatisée de l'architecture optimale d'un réseau de neurones.

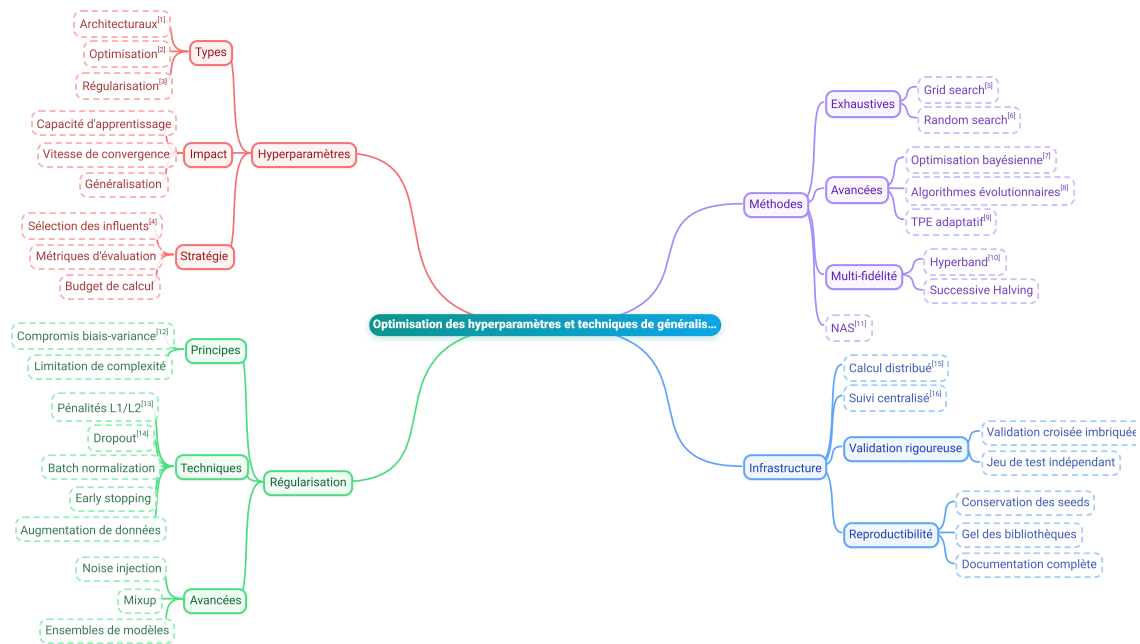
Front de Pareto

Ensemble de solutions optimales où aucun objectif ne peut être amélioré sans en dégrader un autre.

La théorie en Bref

💡 Fondamental

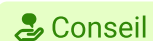
L'**optimisation des hyperparamètres** et la **régularisation** sont deux leviers indissociables pour obtenir des modèles d'IA robustes et performants en production. Les **hyperparamètres** sont des réglages externes fixés avant l'apprentissage (taux d'apprentissage, nombre de couches, batch size, coefficients de régularisation) qui gouvernent la capacité d'apprentissage, la vitesse de convergence et l'aptitude à généraliser. Leur exploration repose sur des méthodes allant de la **recherche par grille** ou **aléatoire** aux approches avancées comme l'**optimisation bayésienne**, les **algorithmes évolutionnaires** ou le **Neural Architecture Search**. La **régularisation** limite le **surapprentissage** en imposant des contraintes sur la complexité du modèle : pénalités **L1/L2**, **dropout**, **batch normalization**, **augmentation de données** ou **early stopping**. Elle agit sur le **compromis biais-variance** pour privilégier des solutions sobres et résilientes. L'efficacité repose sur une démarche rigoureuse : sélection des hyperparamètres influents, validation croisée imbriquée, suivi des métriques, infrastructure automatisée et traçabilité complète des expérimentations.



1. Structure, nombre de couches, activations.
2. Learning rate, batch size.
3. Coefficients L1/L2, dropout.
4. Éviter l'explosion combinatoire.
5. Toutes les combinaisons prédéfinies.
6. Combinaisons aléatoires, plus efficace.
7. Exploration guidée par modèle probabiliste.
8. Population, mutation, sélection.
9. Ajustement dynamique selon résultats.
10. Arrêt précoce des configurations faibles.
11. Recherche automatisée d'architecture neuronale.
12. Équilibre entre sous et surapprentissage.
13. Contrainte sur la norme des poids.
14. Désactivation aléatoire de neurones.
15. Parallélisation des essais, GPU/TPU.
16. Mlflow, Weights & Biases, DVC.

Optimisation des hyperparamètres et techniques de généralisation

Les conseils pro



Comment structurer une stratégie d'optimisation des hyperparamètres efficace ?

- Identifier les hyperparamètres les plus influents pour éviter l'explosion combinatoire et concentrer les ressources.
- Définir des métriques d'évaluation alignées sur les objectifs métier (précision, temps d'inférence, coût).
- Commencer par une exploration large (random search, évolutionnaire) puis affiner sur les zones prometteuses.

- Planifier le budget de calcul et paralléliser les essais pour accélérer le cycle d'expérimentation.

Comment choisir la méthode d'exploration d'hyperparamètres adaptée au contexte ?

- Utiliser grid search uniquement sur des espaces restreints et bien délimités.
- Privilégier random search ou optimisation bayésienne pour des espaces larges ou continus.
- Recourir aux algorithmes évolutionnaires pour des espaces non convexes ou multi-objectifs.
- Appliquer des méthodes multi-fidélité (Hyperband) quand chaque entraînement est coûteux en temps.

Comment sélectionner et doser les techniques de régularisation ?

- Analyser la taille et la qualité du dataset : plus il est petit ou bruité, plus la régularisation est nécessaire.
- Combiner plusieurs techniques complémentaires : dropout, pénalités L1/L2, augmentation de données.
- Surveiller les courbes d'apprentissage pour détecter surapprentissage (écart train/validation) ou sous-apprentissage (performances faibles).
- Adapter dynamiquement l'intensité de régularisation selon les phases d'entraînement ou l'évolution des métriques.

Comment garantir la reproductibilité et la traçabilité des expérimentations ?

- Documenter chaque essai : valeurs d'hyperparamètres, métriques, temps de calcul, environnement.
- Utiliser des outils de suivi centralisés (Mlflow, Weights & Biases, DVC) pour archiver automatiquement les résultats.
- Conserver les seeds aléatoires et geler les versions des bibliothèques utilisées.
- Mettre en place une validation croisée imbriquée et réserver un jeu de test indépendant pour l'évaluation finale.

Comment industrialiser l'optimisation des hyperparamètres dans un workflow de production ?

- Déployer une infrastructure de calcul distribuée pour paralléliser les essais et exploiter GPU/TPU.
- Automatiser la planification des tâches et la réallocation dynamique des ressources aux configurations prometteuses.
- Intégrer l'optimisation dans un pipeline CI/CD pour déclencher automatiquement le tuning à chaque mise à jour de données.
- Établir des bornes raisonnables pour les hyperparamètres afin d'éviter les configurations extrêmes ou instables.

VI Auto-évaluation

Exercice 1 : Hyperparamètres et stratégie d'optimisation

[solution n°1 p. 24]

Exercice

Un hyperparamètre est ajusté automatiquement par le modèle au cours de l'entraînement, au même titre que les poids et les biais d'un réseau de neurones.

Vrai

Faux

Exercice

Pour un réseau de neurones profond, quels hyperparamètres sont systématiquement identifiés comme prioritaires à optimiser en premier ?

Le taux d'apprentissage (learning rate) et le batch size

Le type de base de données et le format des fichiers d'entrée

Le nombre de classes cibles et le nom des colonnes du dataset

La version du framework utilisé et le système d'exploitation du serveur

Exercice

Parmi les éléments suivants, lesquels appartiennent à la catégorie des hyperparamètres architecturaux d'un réseau de neurones ?

A Le nombre de couches du réseau

B Les poids synaptiques appris lors de la rétropropagation

C Le type de fonction d'activation utilisée

D Les biais ajustés automatiquement pendant l'entraînement

E Le nombre de neurones par couche

Exercice

Remettez dans le bon ordre les étapes d'une stratégie d'optimisation des hyperparamètres cohérente, telle que décrite dans la leçon.

A Sélectionner les hyperparamètres les plus influents à ajuster

B Choisir les objectifs d'optimisation (précision, généralisation, temps d'inférence...)

C Planifier l'utilisation des ressources de calcul (nombre d'expériences, parallélisation, temps maximal)

D Fixer les métriques d'évaluation pertinentes (AUC, F1, coût calculatoire...)

Réponse : ____

Exercice 6 : Méthodes d'exploration de l'espace des hyperparamètres

[solution n°2 p. 25]

Exercice

Pour un budget d'essais identique, la recherche par grille (grid search) retrouve systématiquement de meilleures configurations que la recherche aléatoire (random search).

Vrai

Faux

Exercice

Une start-up souhaite optimiser les hyperparamètres d'un assistant vocal en réduisant au maximum le temps total d'expérimentation. Quelle approche est la plus adaptée selon la leçon ?

Une approche séquentielle adaptative de type TPE, qui ajuste dynamiquement les essais en fonction des résultats précédents

Une recherche par grille exhaustive sur toutes les combinaisons possibles

Un test manuel des configurations les plus intuitives, sans protocole formalisé

Une recherche aléatoire pure sur un espace très large, sans affinage progressif

Exercice

Associez chaque méthode d'exploration des hyperparamètres à sa caractéristique principale.

A Random search

B TPE (Tree-structured Parzen Estimator)

C Algorithmes évolutionnaires

D Grid search

Teste toutes les combinaisons prédéfinies, garantit la couverture de l'espace testé mais coûteuse en ressources	Choisit des combinaisons au hasard dans les plages définies, souvent plus efficace pour un même budget d'essais	Font évoluer une population de configurations via héritage, mutation et sélection, efficaces sur des espaces vastes et non convexes	Méthode adaptative qui modélise séparément les zones de bons et mauvais résultats pour diriger les essais suivants
---	---	---	--

Exercice

Quelles affirmations décrivent correctement les approches multi-fidélité comme Hyperband ou Successive Halving ?

A

☐ Elles interrompent l'entraînement précocement pour estimer rapidement le potentiel d'une configuration

B

☐ Elles testent toutes les configurations jusqu'à convergence complète avant de les comparer

C

☐ Seules les configurations les plus prometteuses à un stade intermédiaire accèdent au tour suivant

D

☐ Elles sont particulièrement utiles quand chaque cycle d'entraînement est coûteux en temps

E

☐ Elles garantissent toujours de trouver le meilleur hyperparamètre possible dans l'espace défini

Exercice 11 : Régularisation et contrôle du surapprentissage

[solution n°3 p. 27]

Exercice

Lors du suivi des courbes d'apprentissage d'un modèle, vous observez que les performances sur le jeu d'entraînement et sur le jeu de validation sont toutes deux faibles, avec un écart très réduit entre elles. Quelle interprétation est la plus cohérente avec la leçon ?

☐ Le modèle est probablement sur-régularisé et souffre de sous-apprentissage

☐ Le modèle généralise parfaitement, les deux scores faibles indiquent un problème de données

☐ Le modèle est en situation de surapprentissage classique et nécessite plus de régularisation

Le modèle est optimal : un faible écart entre entraînement et validation est toujours un bon signe

Exercice

Associez chaque technique de régularisation à son mécanisme d'action principal.

A Pénalisation L1/L2

B Early stopping

C Data augmentation

D Dropout

Désactivation aléatoire d'une proportion de neurones à chaque itération pour forcer des représentations plus distribuées	Ajout d'une pénalité sur la norme des poids dans la fonction de coût pour limiter la complexité du modèle	Génération de variations artificielles des données d'entraînement pour élargir l'espace d'apprentissage	Arrêt de l'entraînement dès que les performances sur le jeu de validation cessent de s'améliorer
--	---	---	--

Exercice

Parmi les stratégies suivantes, lesquelles sont décrites dans la leçon comme des approches avancées de lutte contre le surapprentissage exploitant la perturbation volontaire ?

A Mixup : interpolation linéaire entre deux exemples et leurs labels

B Validation croisée imbriquée pour séparer sélection et évaluation finale

C Noise injection : ajout de bruit léger dans les entrées ou les gradients

D Recherche par grille pour couvrir exhaustivement l'espace des hyperparamètres

E Ensembles de modèles combinant les prédictions de plusieurs modèles diversifiés

VII Exercice : Optimiser un modèle d'analyse de sentiment client

Mise en situation

Vous êtes **data scientist** au sein de **SentiFrance**, une entreprise française spécialisée dans l'analyse de la satisfaction client pour le compte de grands groupes (retail, banque, santé, télécoms). Votre équipe a développé un modèle de deep learning basé sur une architecture **transformer** pour analyser le sentiment dans les commentaires clients rédigés en français.

Depuis plusieurs semaines, vous constatez deux problèmes majeurs :

- Le modèle affiche un **accuracy de 96 % sur le jeu d'entraînement**, mais seulement **78 % sur le jeu de validation**, ce qui traduit un **surapprentissage manifeste**.
- Les performances varient fortement selon les secteurs d'activité : **85 % de F1-score sur le retail**, mais seulement **62 % sur la banque** et **68 % sur la santé**.

L'optimisation des hyperparamètres est actuellement réalisée **manuellement** par un membre de l'équipe, qui teste des combinaisons de façon empirique. Ce processus prend en moyenne **3 semaines** par cycle d'optimisation et mobilise **2 GPU A100** en continu.

Données techniques disponibles

Voici les caractéristiques du modèle et de l'infrastructure actuels :

Élément	Valeur actuelle
Architecture	Transformer (6 couches, 8 têtes d'attention)
Dimension cachée	512
Taux d'apprentissage	0.001 (fixe)
Batch size	64
Dropout	0.1 (couche finale uniquement)
Régularisation	Aucune (ni L1, ni L2)
Jeu d'entraînement	120 000 commentaires (60 % retail, 25 % banque, 15 % santé)
Jeu de validation	15 000 commentaires (répartition identique)
Jeu de test	Non constitué
Nombre d'époques	50 (sans early stopping)
Méthode d'optimisation HP	Manuelle, grid search partiel
Budget GPU disponible	4 GPU A100, cluster partagé
Durée maximale souhaitée par cycle	5 jours

Votre responsable vous demande de reprendre en main l'ensemble du pipeline d'optimisation et de régularisation pour améliorer la **généralisation** du modèle, réduire l'**écart de performance inter-secteurs** et **automatiser** le processus de tuning.

Question 1

[solution n°4 p. 29]

Concevez une stratégie d'optimisation des hyperparamètres adaptée à ce modèle NLP. Identifiez les hyperparamètres prioritaires à ajuster, justifiez le choix de la méthode de recherche et précisez comment vous gérez le budget de calcul limité.

Question 2

[solution n°5 p. 30]

Sélectionnez et justifiez les techniques de régularisation à mettre en œuvre pour limiter le surapprentissage constaté. Expliquez comment vous dosez et combinez ces techniques en tenant compte du déséquilibre sectoriel des données.

Question 3

[solution n°6 p. 30]

Proposez une infrastructure et un workflow permettant d'automatiser le processus d'optimisation des hyperparamètres, en intégrant les exigences de traçabilité et de reproductibilité.

Question 4

[solution n°7 p. 31]

Élaborez un plan d'expérimentation structuré pour mesurer l'efficacité des techniques d'optimisation et de régularisation proposées. Précisez les métriques, les comparaisons à effectuer et les critères de décision.

Solutions des exercices

Solution n°1

[exercice p. 18]

Exercice

Un hyperparamètre est ajusté automatiquement par le modèle au cours de l'entraînement, au même titre que les poids et les biais d'un réseau de neurones.

Vrai

✓ Faux



C'est précisément ce qui distingue un hyperparamètre d'un paramètre : les poids et biais sont appris automatiquement à partir des données, tandis que les hyperparamètres (taux d'apprentissage, batch size, profondeur d'arbre...) sont fixés avant l'entraînement par l'ingénieur ou par une stratégie de recherche externe. Confondre les deux conduit à sous-estimer l'effort de configuration nécessaire avant tout entraînement.

Exercice

Pour un réseau de neurones profond, quels hyperparamètres sont systématiquement identifiés comme prioritaires à optimiser en premier ?

✓ Le taux d'apprentissage (learning rate) et le batch size

Le type de base de données et le format des fichiers d'entrée

Le nombre de classes cibles et le nom des colonnes du dataset

La version du framework utilisé et le système d'exploitation du serveur



Le taux d'apprentissage et le batch size sont des hyperparamètres d'optimisation qui influencent directement la vitesse de convergence, la stabilité de l'entraînement et la qualité de la généralisation. Les autres propositions concernent des aspects techniques d'infrastructure ou de données qui ne sont pas des hyperparamètres au sens strict du terme.

Exercice

Parmi les éléments suivants, lesquels appartiennent à la catégorie des hyperparamètres architecturaux d'un réseau de neurones ?

A

Le nombre de couches du réseau

B Les poids synaptiques appris lors de la rétropropagation

C Le type de fonction d'activation utilisée

D Les biais ajustés automatiquement pendant l'entraînement

E Le nombre de neurones par couche

🔍 Les hyperparamètres architecturaux définissent la structure du réseau avant l'entraînement : nombre de couches, type d'activation, nombre de neurones par couche. Les poids synaptiques et les biais, en revanche, sont des paramètres internes appris automatiquement à partir des données — ils ne sont donc pas des hyperparamètres.

Exercice

Remettez dans le bon ordre les étapes d'une stratégie d'optimisation des hyperparamètres cohérente, telle que décrite dans la leçon.

A Choisir les objectifs d'optimisation (précision, généralisation, temps d'inférence...)

B Sélectionner les hyperparamètres les plus influents à ajuster

C Fixer les métriques d'évaluation pertinentes (AUC, F1, coût calculatoire...)

D Planifier l'utilisation des ressources de calcul (nombre d'expériences, parallélisation, temps maximal)

🔍 La leçon décrit une démarche séquentielle : on commence par définir ce que l'on cherche à optimiser (objectifs), puis on identifie les leviers les plus impactants pour éviter l'explosion combinatoire, ensuite on choisit comment mesurer les résultats (métriques), et enfin on planifie les ressources nécessaires. Inverser ces étapes conduit à des campagnes d'optimisation coûteuses et peu ciblées.

Solution n°2

[exercice p. 19]

Exercice

Pour un budget d'essais identique, la recherche par grille (grid search) retrouve systématiquement de meilleures configurations que la recherche aléatoire (random search).

Vrai

✓ Faux



Les travaux de Bergstra & Bengio (2012) cités dans la leçon montrent l'inverse : pour un même budget d'essais, la recherche aléatoire repère souvent les zones optimales plus rapidement que la grille. La raison est que tous les hyperparamètres ne sont pas également sensibles à leur plage de variation — la grille gaspille des ressources en testant des combinaisons peu influentes, tandis que le random search explore plus efficacement les dimensions réellement déterminantes.

Exercice

Une start-up souhaite optimiser les hyperparamètres d'un assistant vocal en réduisant au maximum le temps total d'expérimentation. Quelle approche est la plus adaptée selon la leçon ?

✓ Une approche séquentielle adaptative de type TPE, qui ajuste dynamiquement les essais en fonction des résultats précédents

Une recherche par grille exhaustive sur toutes les combinaisons possibles

Un test manuel des configurations les plus intuitives, sans protocole formalisé

Une recherche aléatoire pure sur un espace très large, sans affinage progressif



La leçon illustre précisément ce cas : une start-up utilisant une approche TPE a réduit son temps d'optimisation de 40 % tout en améliorant les performances. Le TPE modélise séparément les zones de bons et mauvais résultats pour diriger les essais suivants, ce qui le rend bien plus efficace que la grille (trop exhaustive) ou le random search pur (non adaptatif).

Exercice

Associez chaque méthode d'exploration des hyperparamètres à sa caractéristique principale.

Teste toutes les combinaisons prédéfinies, garantit la couverture de l'espace testé mais coûteuse en ressources	Choisit des combinaisons au hasard dans les plages définies, souvent plus efficace pour un même budget d'essais	Font évoluer une population de configurations via héritage, mutation et sélection, efficaces sur des espaces vastes et non convexes	Méthode adaptative qui modélise séparément les zones de bons et mauvais résultats pour diriger les essais suivants
Grid search	Random search	Algorithmes évolutionnaires	TPE (Tree-structured Parzen Estimator)



Chaque méthode répond à un contexte différent : la grid search convient aux petits espaces bien délimités ; le random search est plus efficace dès que certains hyperparamètres sont peu sensibles ; les algorithmes évolutionnaires excellent sur des espaces larges et discontinus ; le TPE est une méthode adaptative qui capitalise sur les résultats précédents pour affiner la recherche de manière séquentielle.

Exercice

Quelles affirmations décrivent correctement les approches multi-fidélité comme Hyperband ou Successive Halving ?



Elles interrompent l'entraînement précocement pour estimer rapidement le potentiel d'une configuration

B

Elles testent toutes les configurations jusqu'à convergence complète avant de les comparer



Seules les configurations les plus prometteuses à un stade intermédiaire accèdent au tour suivant



Elles sont particulièrement utiles quand chaque cycle d'entraînement est coûteux en temps

E

Elles garantissent toujours de trouver le meilleur hyperparamètre possible dans l'espace défini



Les approches multi-fidélité reposent sur un principe de sélection progressive : on évalue rapidement de nombreuses configurations en les interrompant tôt, puis on alloue les ressources complètes uniquement aux candidats les plus prometteurs. Cela permet de diviser le temps de recherche par deux dans des contextes industriels où les entraînements durent plusieurs heures. En revanche, elles ne garantissent pas de trouver l'optimum absolu — elles optimisent l'efficacité de la recherche, pas son exhaustivité.

Solution n°3

[exercice p. 20]

Exercice

Lors du suivi des courbes d'apprentissage d'un modèle, vous observez que les performances sur le jeu d'entraînement et sur le jeu de validation sont toutes deux faibles, avec un écart très réduit entre elles. Quelle interprétation est la plus cohérente avec la leçon ?

✓ Le modèle est probablement sur-régularisé et souffre de sous-apprentissage

Le modèle généralise parfaitement, les deux scores faibles indiquent un problème de données

Le modèle est en situation de surapprentissage classique et nécessite plus de régularisation

Le modèle est optimal : un faible écart entre entraînement et validation est toujours un bon signe



La leçon précise explicitement ce signal : si le gap entraînement/validation est faible mais que les deux performances sont mauvaises, c'est souvent un indicateur de sous-apprentissage lié à une régularisation excessive. Le modèle est devenu trop contraint pour capter les tendances importantes. Ce cas se distingue du surapprentissage, où les performances d'entraînement sont élevées mais celles de validation s'effondrent.

Exercice

Associez chaque technique de régularisation à son mécanisme d'action principal.

Désactivation aléatoire d'une proportion de neurones à chaque itération pour forcer des représentations plus distribuées	Ajout d'une pénalité sur la norme des poids dans la fonction de coût pour limiter la complexité du modèle	Génération de variations artificielles des données d'entraînement pour élargir l'espace d'apprentissage	Arrêt de l'entraînement dès que les performances sur le jeu de validation cessent de s'améliorer
Dropout	Pénalisation L1/L2	Data augmentation	Early stopping



Chaque technique agit sur un levier différent : le dropout introduit de la stochasticité dans la structure du réseau ; la pénalisation L1/L2 contraint directement les poids via la fonction de coût ; la data augmentation enrichit artificiellement les données pour éviter la mémorisation ; l'early stopping contrôle la dynamique d'entraînement en stoppant avant la surexploitation. Comprendre ces mécanismes distincts est essentiel pour choisir la bonne technique selon le contexte.

Exercice

Parmi les stratégies suivantes, lesquelles sont décrites dans la leçon comme des approches avancées de lutte contre le surapprentissage exploitant la perturbation volontaire ?

A

Mixup : interpolation linéaire entre deux exemples et leurs labels

B

Validation croisée imbriquée pour séparer sélection et évaluation finale

C

Noise injection : ajout de bruit léger dans les entrées ou les gradients

D Recherche par grille pour couvrir exhaustivement l'espace des hyperparamètres

E Ensembles de modèles combinant les prédictions de plusieurs modèles diversifiés

🔍 La leçon regroupe sous 'stratégies avancées exploitant la perturbation volontaire' : le mixup (qui lisse les frontières de décision par interpolation), le noise injection (qui perturbe les entrées ou gradients) et les ensembles de modèles (qui réduisent la variance par diversification). La validation croisée imbriquée est une bonne pratique d'évaluation, et la grid search est une méthode d'exploration des hyperparamètres — ni l'une ni l'autre ne relèvent de la perturbation volontaire.

[exercice p. 23] **Solution n°4**

Réponse attendue

La stratégie d'optimisation doit cibler en priorité les hyperparamètres les plus influents sur un transformer NLP : le **taux d'apprentissage** (actuellement fixe à 0.001, trop élevé pour un fine-tuning), le **batch size**, le **taux de dropout** (actuellement trop faible et limité à une seule couche), et le **coefficient de régularisation L2** (actuellement absent). Le nombre de couches et de têtes d'attention constituent un second palier d'exploration. Compte tenu du budget limité à **4 GPU sur 5 jours**, une recherche par grille exhaustive est exclue. La méthode recommandée est une approche en deux phases : d'abord un **random search** large pour identifier les zones prometteuses de l'espace, puis une **optimisation bayésienne (TPE)** pour affiner les combinaisons dans ces zones. L'utilisation d'un protocole **Hyperband** ou **Successive Halving** permet d'éliminer précocement les configurations peu performantes après quelques époques, réduisant ainsi le coût computationnel global tout en conservant la qualité de la recherche.

Éléments de solution

- **Hyperparamètres prioritaires à ajuster :**
 - Taux d'apprentissage : plage [1e-5, 1e-3] avec scheduler (warm-up + décroissance cosinus)
 - Dropout : plage [0.1, 0.5], appliqué à plusieurs couches (attention, feed-forward, embeddings)
 - Weight decay (L2) : plage [1e-4, 1e-2]
 - Batch size : tester 16, 32, 64 (impact sur la généralisation et la stabilité)
- **Hyperparamètres secondaires :** nombre de couches (4 à 8), nombre de têtes d'attention (4 à 12), dimension cachée (256 à 768)
- **Méthode de recherche justifiée :** random search initial (50-100 essais) → TPE bayésien sur les 20 % de configurations les plus prometteuses
- **Gestion du budget :** Successive Halving avec arrêt précoce après 5 époques pour les configurations sous-performantes, parallélisation sur les 4 GPU disponibles
- **Rappel méthodologique :** l'écart de **18 points** entre entraînement (96 %) et validation (78 %) confirme un surapprentissage sévère — l'optimisation des hyperparamètres seule ne suffira pas sans régularisation
- **Feedback :** une erreur fréquente consiste à explorer trop d'hyperparamètres simultanément sans hiérarchiser — commencez par les 3-4 leviers majeurs avant d'élargir

[exercice p. 23] **Solution n°5****Réponse attendue**

Face à un écart de **18 points** entre entraînement et validation et un déséquilibre sectoriel marqué (60 % retail, 25 % banque, 15 % santé), plusieurs techniques de régularisation doivent être combinées. Le **dropout** doit être étendu à l'ensemble des couches du transformer (attention, feed-forward, embeddings) avec un taux relevé entre **0.2 et 0.4**. Une **régularisation L2 (weight decay)** doit être introduite pour pénaliser les poids excessifs. L'**early stopping** est indispensable : l'entraînement sur 50 époques sans arrêt précoce favorise la mémorisation. Un monitoring de la loss de validation avec une patience de 5 à 10 époques permettra d'interrompre l'entraînement au moment optimal. Pour le déséquilibre sectoriel, une **augmentation de données ciblée** sur les secteurs sous-représentés (banque et santé) — par paraphrase, back-translation ou substitution de synonymes — renforcera la diversité des exemples. Enfin, la **layer normalization**, déjà standard dans les transformers, doit être vérifiée et complétée par un **label smoothing** (facteur 0.1) pour éviter des prédictions trop tranchées.

Éléments de solution

- **Dropout étendu** : appliquer à toutes les couches (pas uniquement la couche finale), taux entre 0.2 et 0.4 — justification : le dropout sur une seule couche laisse le reste du réseau libre de surapprendre
- **Weight decay (L2)** : coefficient entre 1e-4 et 1e-2, intégré directement dans l'optimiseur (AdamW recommandé)
- **Early stopping** : patience de 5 à 10 époques sur la loss de validation, avec sauvegarde du meilleur checkpoint
- **Augmentation de données** :
 - Back-translation (FR → EN → FR) pour les secteurs banque et santé
 - Paraphrase via modèle génératif
 - Objectif : rééquilibrer la représentation sectorielle vers une distribution plus homogène
- **Label smoothing** (facteur 0.1) : réduit la confiance excessive du modèle et améliore la calibration
- **Layer normalization** : vérifier son application systématique dans chaque bloc transformer
- **Rappel du compromis biais-variance** : l'objectif n'est pas de supprimer tout écart entraînement/validation, mais de le ramener à **3-5 points** tout en maintenant une performance absolue satisfaisante
- **Feedback** : attention à la sur-régularisation — si les performances d'entraînement ET de validation chutent simultanément, c'est un signal de sous-apprentissage lié à un excès de contraintes

[exercice p. 23] **Solution n°6****Réponse attendue**

L'infrastructure d'automatisation doit s'articuler autour de trois composantes. Premièrement, un **framework d'optimisation** comme Optuna (avec sampler TPE et pruner Hyperband) ou Ray Tune, capable de distribuer les essais sur les **4 GPU A100** disponibles et d'appliquer l'arrêt précoce automatique. Deuxièmement, un **outil de tracking d'expériences** comme MLflow ou Weights & Biases pour enregistrer automatiquement chaque essai : hyperparamètres testés, métriques obtenues (accuracy, F1 global et par secteur), courbes d'apprentissage, temps de calcul et environnement d'exécution. Troisièmement, un **pipeline reproductible** intégrant le versioning des

données (DVC), le gel des dépendances (fichier requirements.txt ou environnement conda), la fixation des seeds aléatoires et l'archivage des scripts de configuration. Ce workflow doit permettre de passer d'un cycle manuel de **3 semaines** à un cycle automatisé de **5 jours maximum**, tout en garantissant qu'un audit ou un collègue puisse reproduire exactement chaque expérience.

Éléments de solution

- **Framework d'optimisation recommandé** : Optuna (léger, flexible, intégration native TPE + Hyperband) ou Ray Tune (meilleure scalabilité multi-GPU)
- **Tracking des expériences** :
 - MLflow ou Weights & Biases pour le logging automatique
 - Métriques à suivre : loss, accuracy, F1-score global, F1 par secteur (retail, banque, santé), temps par essai
 - Sauvegarde des courbes d'apprentissage (train vs validation) pour chaque configuration
- **Reproductibilité** :
 - Versioning des données avec DVC
 - Fixation des seeds (random, numpy, torch)
 - Gel des versions de bibliothèques
 - Archivage des fichiers de configuration YAML/JSON pour chaque campagne
- **Orchestration** : planification des jobs avec un scheduler (SLURM ou Kubernetes) pour optimiser l'utilisation des 4 GPU
- **Jeu de test sanctuarisé** : constituer un jeu de test (absent actuellement) de **10 000 à 15 000 commentaires**, équilibré par secteur, utilisé une seule fois en fin de processus
- **Feedback** : l'absence de jeu de test dans la configuration actuelle est un risque majeur – sans évaluation finale indépendante, il est impossible de garantir que l'optimisation n'a pas surajusté le modèle aux données de validation elles-mêmes

[exercice p. 23] **Solution n°7**

Réponse attendue

Le plan d'expérimentation doit suivre une démarche en **quatre phases**. **Phase 1 – Baseline** : figer le modèle actuel (sans modification) et enregistrer toutes les métriques de référence sur les jeux d'entraînement, de validation et sur un jeu de test nouvellement constitué. **Phase 2 – Régularisation isolée** : appliquer les techniques de régularisation (dropout étendu, weight decay, early stopping, augmentation de données) sans modifier les hyperparamètres, pour mesurer leur impact propre sur l'écart entraînement/validation et sur le F1 par secteur. **Phase 3 – Optimisation des hyperparamètres** : lancer le pipeline automatisé (random search + TPE + Hyperband) avec les régularisations activées, pour identifier la meilleure configuration globale. **Phase 4 – Validation finale** : évaluer la configuration retenue sur le jeu de test sanctuarisé, comparer les résultats à la baseline et vérifier que l'écart de performance inter-secteurs a été réduit. Les critères de décision sont : un écart entraînement/validation inférieur à **5 points**, un F1-score minimum de **75 %** sur chaque secteur, et un temps d'inférence compatible avec les contraintes de production.

Éléments de solution

- **Phase 1 – Baseline documentée :**
 - Métriques : accuracy, F1-score global, F1 par secteur, écart train/val, temps d'entraînement
 - Constitution du jeu de test : 10 000-15 000 commentaires, équilibré par secteur
 - Enregistrement dans MLflow/W&B comme point de référence
- **Phase 2 – Ablation de la régularisation :**
 - Tester chaque technique isolément (dropout seul, weight decay seul, early stopping seul, augmentation seule)
 - Puis tester les combinaisons progressives
 - Mesurer l'impact sur l'écart train/val et le F1 par secteur
- **Phase 3 – Optimisation automatisée :**
 - 100-150 essais avec random search + TPE
 - Successive Halving pour l'arrêt précoce des configurations faibles
 - Suivi du front de Pareto : F1 global vs temps d'inférence vs écart inter-secteurs
- **Phase 4 – Validation finale :**
 - Évaluation unique sur le jeu de test sanctuarisé
 - Tests de robustesse : ajout de bruit, commentaires hors domaine, données dégradées
- **Critères de décision :**
 - Écart train/val < **5 points**
 - F1-score $\geq$ **75 %** sur chaque secteur individuellement
 - Temps d'inférence < **100 ms** par commentaire
 - Reproductibilité confirmée (2 runs identiques à ± 1 %)
- **Feedback :** l'approche par ablation (phase 2) est essentielle — elle permet de comprendre la contribution de chaque technique et d'éviter d'empiler des régularisations dont l'effet combiné pourrait être contre-productif